

ENTREGA DE TAREAS

Sprint 1: Vistas — FutbolWebApp

Pablo Arrazola Trebol

Prácticas — Data Control

24/10/2026

Resumen

Se ha creado la base de datos futbol en MySQL a partir del script proporcionado y el workspace FutbolWebApp con un proyecto Web Mobile del mismo nombre en DataFlex. Se ha conectado el proyecto a MySQL mediante una conexión gestionada, se han migrado a MySQL las tablas embebidas que trae el framework y se han conectado las tablas de futbol. Después se han creado las vistas Select y Zoom de liga, equipo, entrenador, jugador y posicion, y las tarjetas (tiles) de acceso en Dashboard.wo. Todo el trabajo se ha versionado con Git siguiendo gitflow.

1. Base de datos futbol en MySQL

El script entregado crea la base de datos con el nombre football; se ha cambiado a futbol, como pide el enunciado, en las sentencias CREATE DATABASE y USE. El resto del script no se modifica.

```
CREATE DATABASE IF NOT EXISTS futbol /*!40100 DEFAULT CHARACTER SET utf8mb4
    COLLATE utf8mb4_0900_ai_ci */;
USE futbol;
```

Se ejecuta en MySQL Workbench (File > Run SQL Script) o por consola:

```
mysql -u root -p < futbol.sql
```

Estructura resultante:

Tabla	Campos	Relación
liga	ID, Nombre, Pais, Equipos, EquiposMax	—
equipo	ID, Nombre, LigaID	LigaID → liga.ID
entrenador	ID, Nombre, Apellido, Nacionalidad, EquipoID	EquipoID → equipo.ID (único: un entrenador por equipo)
jugador	ID, Nombre, Apellido, Nacionalidad, EquipoID	EquipoID → equipo.ID
posicion	ID, Nombre	—
jugadorposicion	ID, JugadorID, PosicionID	Tabla intermedia jugador ↔ posicion (sin vistas en este sprint)

Datos de partida: 2 ligas, 6 equipos, 7 entrenadores (uno sin equipo), 60 jugadores y 6 posiciones. Todas las tablas tienen clave primaria ID e índices únicos adicionales (por ejemplo Apellido + ID), necesarios para que DataFlex pueda buscar y ordenar por esos campos.

Captura: esquema futbol en MySQL Workbench con sus tablas:

2. Workspace y proyecto FutbolWebApp

1. En DataFlex Studio: File > New > Workspace, nombre **FutbolWebApp**.
2. Dentro del workspace: File > New > Project > **Mobile Web Project**, nombre **FutbolWebApp**. El wizard genera FutbolWebApp.src, el objeto cWebApp con estilo Mobile/Touch, el login, el menú y Dashboard.wo, y registra la aplicación en IIS.
3. Compilar y ejecutar (F5) para comprobar que la aplicación vacía arranca en el navegador.

Captura: workspace FutbolWebApp con el proyecto en el Workspace Explorer:

3. Conexión de la BD futbol con el proyecto

Requisito: tener instalado el MySQL Connector/ODBC con la misma arquitectura (32 o 64 bits) que DataFlex. La conexión se hace con el driver ODBC de DataFlex.

1. Crear una **conexión gestionada** (Managed Connection) en el Studio, con servidor 127.0.0.1, puerto 3306, base de datos futbol, usuario y contraseña de MySQL.
2. Probar la conexión (Test/Login) antes de continuar.
3. La conexión queda guardada en DFConnId.ini del workspace y los ficheros .int de cada tabla la referencian por su ID, en lugar de llevar el servidor y la contraseña escritos.

Captura: conexión gestionada a futbol creada y probada:

4. Migrar las tablas embebidas del framework a MySQL

El proyecto web nuevo trae tablas en la base de datos embebida de DataFlex que usa el framework (usuarios WebAppUser, sesiones WebAppSession y propiedades de servidor WebAppServerProps, entre otras). Se llevan a MySQL con el **SQL Conversion Wizard** del Studio:

1. Abrir el SQL Conversion Wizard y elegir el driver ODBC y la conexión gestionada de futbol.
2. Seleccionar todas las tablas embebidas del workspace.

3. Convertir: el wizard crea las tablas en MySQL, copia los datos y cambia las entradas del filelist para que apunten a MySQL (ficheros .int).
4. Comprobar en MySQL Workbench que las tablas webapp* aparecen ya dentro de futbol y que el login de la aplicación sigue funcionando.

Captura: tablas del framework ya en MySQL (Workbench) o en el Table Explorer con driver ODBC:

5. Traer las tablas de MySQL a DataFlex

1. Abrir el **SQL Connect/Repair Wizard**, elegir el driver ODBC y la conexión gestionada.
2. Marcar las tablas en estado «Not Connected»: liga, equipo, entrenador, jugador, posicion y jugadorposicion.
3. Conectar: se añaden al filelist y se crea un .int por tabla con campos, índices y conexión.
4. En el Table Editor, revisar las **relaciones** (equipo.LigaID → liga.ID, entrenador.EquipoID y jugador.EquipoID → equipo.ID, jugadorposicion → jugador y posicion) y crearlas si no se han importado.
5. Crear el **Data Dictionary** de cada tabla (cLigaDataDictionary, cEquipoDataDictionary, etc.) con sus relaciones padre-hijo y, como mínimo, Nombre y Apellido como obligatorios.

Captura: Table Explorer con las tablas de futbol conectadas:

6. Vistas Select y Zoom

Para cada tabla se crean dos vistas con File > New > Web Object: **Mobile Select View** (lista con cWebList, peViewType vtSelect) y **Mobile Zoom View** (formulario con cWebForm, peViewType vtZoom). Se crean en orden de padre a hijo para que las búsquedas de padre ya existan.

Tabla	Select (columnas de la lista)	Zoom (campos del formulario)
liga	Nombre, País, Equipos	Nombre, País, Equipos, EquiposMax
equipo	Nombre, Liga	Nombre, Liga (búsqueda en SelectLiga)
entrenador	Nombre, Apellido, Equipo	Nombre, Apellido, Nacionalidad,

Tabla	Select (columnas de la lista)	Zoom (campos del formulario)
		Equipo (búsqueda en SelectEquipo)
jugador	Nombre, Apellido, Equipo, Nacionalidad	Nombre, Apellido, Nacionalidad, Equipo (búsqueda en SelectEquipo)
posicion	Nombre	Nombre

Navegación drilldown: al pulsar una fila de la Select se abre su Zoom. En el Zoom, el campo del padre (por ejemplo Equipo) navega a la Select del padre; al elegir una fila se vuelve al Zoom con el valor seleccionado. Ejemplo de la Select de jugador:

```
Procedure OnRowClick String sRowID
   tWebNavigateData NavigateData
   Get GetNavigateData to NavigateData
   If (NavigateData.eNavigateType = nfFromChild) Begin
      Send NavigateClose Self              // devolver selección al Zoom
   End
   Else Begin
      Send NavigateForward of oZoomJugador Self
   End
End_Procedure
```

El código completo de referencia de SelectJugador.wo y ZoomJugador.wo se adjunta en SelectJugador_ZoomJugador_referencia.txt.

Captura: SelectJugador en el navegador:

Captura: ZoomJugador de un jugador:

7. Tiles en Dashboard.wo

En Dashboard.wo se sustituye el grupo de tarjetas de ejemplo por cinco tarjetas en una cuadrícula de 12 columnas (tres de 4 columnas y dos de 6). Cada tarjeta es un cWebHtmlBox con la clase CSS Tile; al hacer clic llama a OnClick y navega a la Select de su tabla.

```
Object oTiles_grp is a cWebGroup
   Set pbShowBorder to False
   Set piColumnCount to 12
   Set psCSSClass to "TilesGroup"

   // ---- Fila 1: tres tarjetas de 4 columnas ----
```

```
    Object oTileLiga is a cWebHtmlBox
        Set piColumnSpan to 4
        Set psCSSClass to "Tile"
        Set pbServerOnClick to True
        Set psHtml to '<div class="Tile_BigLabel" data-ServerOnClick="tile"
data-OnClickParam="liga">Ligas</div><div class="Tile_SmallLabel">Competiciones</div>'

        Procedure OnClick String sId String sParam
            Send NavigateForward of oSelectLiga Self
        End_Procedure
    End_Object
    // ... oTileEquipo, oTileEntrenador, oTileJugador y oTilePosicion igual
End_Object
```

El HTML de la tarjeta lleva los atributos data-ServerOnClick y data-OnClickParam, que son los que disparan el evento OnClick en el servidor (pbServerOnClick = True). Las rutas se registran con WebRegisterPath ntNavigateForward. El código completo está en Dashboard_tiles.txt.

Captura: Dashboard con las cinco tarjetas:

8. Git y GitHub (gitflow)

Rama	Uso
main	Producto final; solo recibe merges de develop al cerrar el sprint (tag v1.0-sprint1)
develop	Versión funcional donde se van integrando las features
feature/conexion-mysql	Conexión gestionada, migración de tablas embebidas, tablas y DDs
feature/vista-liga … feature/vista-posicion	Una rama por vista (Select + Zoom de cada tabla)
feature/dashboard-tiles	Tarjetas del Dashboard
feature/mejoras-visuales	Opcional: ajustes de estilo

Ciclo de cada funcionalidad:

```
git checkout -b feature/vista-jugador develop
# ... trabajo en el Studio, compilar y probar ...
git add -A
git commit -m "Vistas Select y Zoom de jugador"
git checkout develop
git merge --no-ff feature/vista-jugador -m "Merge feature/vista-jugador"
```

Cierre del sprint:

```
git checkout main
git merge --no-ff develop -m "Release Sprint 1: vistas y dashboard"
git tag -a v1.0-sprint1 -m "Sprint 1 - Vistas"
```

```
git push origin --all
git push origin --tags
```

Se usa merge --no-ff para que cada feature quede como un bloque visible en el historial. El .gitignore excluye los compilados (.exe, .prp, .prn, .pbg, .dep, .dbg). Los comandos completos están en git_workflow.txt.

Captura: historial de ramas en Sourcetree o git log --graph:

Captura: repositorio FutbolWebApp en GitHub con main, develop y las ramas feature:

Observaciones

- La tabla posicion trae dos registros de prueba (Pepito y Manolito) además de las cuatro posiciones reales; se han dejado tal cual vienen en el script.
- El entrenador Alvaro Arbeloa no tiene equipo (EquipoID NULL); la vista lo muestra con el equipo vacío.
- La restricción única de entrenador.EquipoID impide asignar dos entrenadores al mismo equipo: MySQL devolverá error al guardar desde el Zoom.